

Оглавление

1. Введение.....	3
2. Предварительные сведения.....	4
2.1. Определения и обозначения.....	4
2.2. Быстрое возведение в степень.....	5
2.3. Дискретное преобразование Фурье.....	5
2.4. Криптосистемы, основанные на QC-MDPC кодах.....	6
2.5. QC-MDPC Декодер.....	8
3. Циклических свертки в $\frac{F_p[x]}{x^n-1}$	9
3.1. Разреженные свертки.....	10
3.2. Быстрое преобразование Фурье.....	10
4. Обращения многочленов в $\frac{F_p[x]}{x^n-1}$	11
4.1. Расширенный алгоритм Евклида в $\mathbb{F}_q[x]$	11
4.2. Нахождение обратного элемента через малую теорему Ферма.....	13
4.3. Сравнение с расширенным алгоритмом Евклида.....	14
5. Оптимизация циклических свертки в $\frac{F_4[x]}{x^n-1}$	14
6. Оптимизация обращений многочленов из $\frac{F_4[x]}{x^n-1}$	15
7. Результаты численных экспериментов.....	16
8. Выводы.....	17

1. Введение

В 2024 году завершилось соревнование «Post-Quantum Cryptography Standartization», проводимое NIST [3]. Целью соревнования было включить в список стандартов NIST новые криптосистемы, устойчивые к атакам со стороны квантовых компьютеров. В частности, в отборе участвовали механизмы инкапсуляции ключей.

Механизм инкапсуляции ключей (KEM, Key Encapsulation Mechanism) — это набор алгоритмов, которые могут использоваться двумя сторонами для безопасного установления общего секретного ключа через публичный канал [5]. Общий секретный ключ затем может быть использован с традиционными симметричными криптографическими алгоритмами для, например, шифрования или аутентификации.

В четвертый раунд соревнования прошли четыре KEM: BIKE, Classic McEliece, HQC, SIKE [4]. В данной работе рассматривается BIKE — криптосистема, основанная на QC-MDPC кодах [7]. По итогам четвёртого раунда NIST отдал предпочтение HQC как резервному стандарту, отметив, что анализ частоты ошибок декодирования (DFR) для BIKE пока менее зрелый [6]. Тем не менее BIKE остаётся активно исследуемой криптосистемой: она обеспечивает наименьший размер ключей среди всех кандидатов и допускает эффективную программную реализацию.

Структура работы. Раздел 2 вводит определения и обозначения, используемые в дальнейшем. В разделе 3 будут показаны общие подходы к вычислению циклических сверток, а в разделе 4 — общие подходы к обращению многочленов. В разделе 5 описана оптимизированная реализация циклических сверток над F_4 . В разделе 6 будет показано, как перенести оптимизированный алгоритм обращения многочлена для F_2 на поле F_4 . Раздел 7 содержит подробности о программной реализации и результаты вычислительных экспериментов. Выводы из проведенной работы содержатся в разделе 8.

2. Предварительные сведения

2.1. Определения и обозначения

Через F_p обозначим конечное поле мощности p , где p – простое число. Через $F_p[x]$ обозначим кольцо многочленов с коэффициентами из этого поля. Пусть $f(x) \in F_p[x]$ – многочлен степени n . Через $R = \frac{F_p[x]}{f(x)}$ обозначим *факторкольцо многочленов* по идеалу, порожденному f . Оно состоит из многочленов степени меньше n с коэффициентами из F_p , а умножение выполняется по модулю $f(x)$. В криптосистемах на QC-MDPC кодах используется $f(x) = x^n - 1$, так что $x^n \equiv 1$. Из-за этого умножение многочленов в этих криптосистемах может быть реализовано *циклической сверткой*.

Циклическая свёртка двух последовательностей $x[n]$ и $h[n]$ длины N определяется как

$$(x \otimes h)[n] = \sum_{m=0}^{N-1} x[m] h[(n - m) \bmod N], \quad (1)$$

Линейным $[n, k]_q$ кодом называют k -мерное подпространство F_q^n . Элементы кода называются кодовыми словами. Линейный код можно задать проверочной матрицей $H \in F_q^{(n-k) \times n}$: элемент $c \in F_q^n$ является кодовым словом тогда и только тогда, когда $Hc^T = 0$.

Поле $F_4 = \{0, 1, \alpha, \alpha + 1\}$ является расширением F_2 степени 2, задаваемым неприводимым многочленом $x^2 + x + 1$. Для хранения элементов из F_4 нужны всего 2 бита, что позволяет выполнять операции над большим количеством элементов одновременно с использованием побитовых операций.

Малая теорема Ферма. В конечном поле F_{q^m} для любого ненулевого элемента a выполняется $a^{q^m-1} = 1$.

Китайская теорема об остатках. Если $f(x) = f_1(x) \cdots f_l(x)$, где многочлены f_i попарно взаимно просты, то

$$\frac{\mathbb{F}_q[x]}{f(x)} \cong \frac{\mathbb{F}_q[x]}{f_1} \oplus \frac{\mathbb{F}_q[x]}{f_2} \oplus \cdots \oplus \frac{\mathbb{F}_q[x]}{f_l} \quad (2)$$

2.2. Быстрое возведение в степень

Для возведения многочлена в степень будет использоваться алгоритм быстрого возведения в степень.

Algorithm 1 Быстрое возведение в степень

```

1:  $e \leftarrow S$ 
2:  $result \leftarrow 1$ 
3:  $base \leftarrow a$ 
4: while  $e > 0$  do
5:   if  $e$  нечётно then
6:      $result \leftarrow result \cdot base \bmod (x^n - 1)$ 
7:   end if
8:    $base \leftarrow base^2 \bmod (x^n - 1)$ 
9:    $e \leftarrow \lfloor e/2 \rfloor$ 
10: end while
11: return  $result$ 

```

2.3. Дискретное преобразование Фурье

Пусть $x[n]$, $n = 0, 1, \dots, N - 1$, — конечная последовательность комплексных чисел. *Дискретным преобразованием Фурье* (ДПФ) называется отображение

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn}, \quad k = 0, 1, \dots, N - 1, \quad (3)$$

где $W_N = e^{-j2\pi/N}$ — примитивный корень степени N из единицы. Обратное ДПФ (ОДПФ) определяется формулой

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] W_N^{-kn}, \quad n = 0, 1, \dots, N-1. \quad (4)$$

Прямое вычисление требует $\mathcal{O}(N^2)$ арифметических операций. Алгоритмы быстрого преобразования Фурье сокращают эту сложность до $\mathcal{O}(N \log N)$, что делает ДПФ практически применимым при больших N .

Важным практическим свойством ДПФ возможность вычислять с его помощью циклические свертки. Результат циклической свертки можно вычислить как поточечное произведение спектров:

$$\mathcal{F}\{x \circledast h\}[k] = X[k] \cdot H[k], \quad (5)$$

где $X[k]$ и $H[k]$ - ДПФ последовательностей $x[n]$ и $h[n]$ соответственно. Таким образом, циклическая свёртка вычисляется за три шага:

$$x \circledast h = \mathcal{F}^{-1}(\mathcal{F}(x) \cdot \mathcal{F}(h)), \quad (6)$$

что при использовании быстрого преобразования Фурье требует $\mathcal{O}(N \log N)$ операций вместо $\mathcal{O}(N^2)$ при прямом вычислении.

2.4. Криптосистемы, основанные на QC-MDPC кодах

MDPC (Medium density parity check) код — это линейный $[n, k]_q$ код, у которого проверочная матрица H имеет вес строк $\omega \approx \sqrt{n}$.

Говорят, что квадратная матрица A является циркулянтном, если в ней каждая строка получается циклическим сдвигом предыдущей строки вправо:

$$A = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{pmatrix} \quad (7)$$

Из-за такой структуры вместо хранения всей матрицы можно хранить только её первую строку. Матрицам циркулянтам A B можно сопоставить многочлен... Если представить эту строку как многочлен, то результат умножения матриц-циркулянтов будет таким же, как умножение в факторкольце соответствующих многочленов.

MDPC код, обладающий блочно-циркулянтной проверочной матрицей, называется квази-циклическим (QC-MDPC, Quasi-Cyclic Medium Density Parity Check).

$$H = \left(\begin{array}{ccc|ccc} 1 & 0 & 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 \end{array} \right) \quad (8)$$

Рассмотрим, как в криптосистеме на QC-MDPC кодах реализуются эти операции.

Секретный ключ H — проверочная матрица случайного QC-MDPC $[n = ln', (l - 1)n']_q$ -кода, представленная как

$$H = \left(H_1 \mid H_2 \mid \dots \mid H_l \right), \quad (9)$$

где каждый циркулянтный блок H_i размера $n' \times n'$ имеет вес строки γ (равный примерно $\sqrt{n}$). Параметр n' — простое число. В криптосистеме, разработанной в этой работе параметр $l = 2$, и дальше всегда будет предполагаться что $l = 2$. Публичный ключ $\tilde{H}$ — систематическая форма H :

$$\tilde{H} = H_1^{-1}H = \left(I_{n'} \mid H_1^{-1}H_2 \mid \dots \mid H_1^{-1}H_l \right). \quad (10)$$

Поскольку произведение матриц-циркулянтов есть матрица-циркулянт, $\tilde{H}$ также является блочно-циркулянтной. Публичный ключ представляется первыми строками блоков $H_1^{-1}H_i$, $i \in \{2, \dots, l\}$. Таким образом, при $l = 2$, публичный ключ является многочленом степени $n' - 1$.

Общий секрет — вектор ошибок $e \in \mathbb{F}_q^n$ веса t , а шифртекст — его синдром $\tilde{s} = \tilde{H}e^T$. Для расшифрования вычисляется частный синдром $s = He^T = H_1\tilde{s}^T$, который подаётся на вход MDPC-декодеру.

В VIKЕ параметр l выбран равным 2, и все операции реализованы над $\mathbb{F}_2$. Оптимизированная реализация за константное время представлена в [8]. В данной работе разрабатываются быстрые алгоритмы, поддерживающие поле $\mathbb{F}_4$, а также произвольные поля $\mathbb{F}_p$ для простого p .

Шифрование с помощью КЕМ КЕМ реализует три алгоритма: **KeyGen**, **Encaps** и **Decaps**. Алиса генерирует публичный и приватный ключ с помощью **KeyGen** и отправляет публичный ключ Бобу. Боб генерирует общий секретный ключ и шифрует его публичным ключом через **Encaps**, затем отправляет полученный шифртекст Алисе. Алиса расшифровывает его приватным ключом через **Decaps** и получает тот же общий секрет. Таким образом, участники договорились об общем секрете, не передавая его явно по открытому каналу.

2.5. QC-MDPC Декодер

Для декодирования шифртекста в криптосистемах MDPC используется итеративный декодер, восстанавливающий вектор ошибок e по синдрому s и матрице H . Для QC-MDPC кодов применяются декодеры класса *symbol flipping*, которые являются расширением bit-flipping декодера.

Bit-flipping decoder На каждой итерации декодера для каждой позиции $j \in [0, n - 1]$ вычисляется счётчик

$$\sigma_j = |\{i : H_{ij} = 1, s_i = 1\}| \quad (11)$$

Вычисленное σ_j — количество проверочных уравнений, которым позиция j не удовлетворяет. Если σ_j превышает пороговое значение *threshold*, бит e_j переключается и синдром пересчитывается. Итерации повторяются до обнуления синдрома или до достижения максимального числа итераций.

3. Циклических свертки в $\frac{F_p[x]}{x^n-1}$

Самой частой операцией в криптосистеме является циклическая свертка. Напомним, что матрицы циркулянты, которые являются основными объектами в QC-MDPC криптосистеме, могут быть представлены своей первой строкой, как многочлен из факторкольца $\frac{\mathbb{F}_q[x]}{x^n-1}$. В свою очередь умножение многочленов из этого факторкольца эквивалентно циклической свертке из-за $x^n \equiv 1$. Так как эта операция выполняется часто, имеет смысл применить к ней несколько базовых оптимизаций.

Algorithm 2 Циклическая свёртка $c = a \circledast b$

```

1: for каждой позиции  $i$  do
2:   for каждой позиции  $j$  do
3:      $c_i \leftarrow c_i + a_j b_{(i-j) \bmod n}$ 
4:   end for
5: end for
```

Операция взятия остатка от деления вычислительно дороже чем обычные арифметические операции. От неё можно просто избавиться, расщепив цикл на два:

Algorithm 3 Циклическая свёртка $c = a \circledast b$ без взятия остатка от деления

```
1: for каждой позиции  $i$  do
2:   for каждой позиции  $j \leq i$  do
3:      $c_i \leftarrow c_i + a_j b_{i-j}$ 
4:   end for
5:   for  $j$  от  $i + 1$  до  $n - 1$  do
6:      $c_i \leftarrow c_i + a_j b_{n+(i-j)}$ 
7:   end for
8: end for
```

3.1. Разреженные свертки

Напомним, что секретный ключ криптосистемы является матрицей с весом строки γ , который примерно равен $\sqrt{n}$. Это означает, что многочлен, описывающий эту матрицу, разреженный. Используя этот факт, можно выполнять операции только для ненулевых позиций:

Algorithm 4 Циклическая свёртка $c = a \circledast b$, где a – разреженный многочлен

```
1: for каждой позиции  $i$  do
2:   for каждой позиции  $j$  такой, что  $a_j \neq 0$  do
3:      $c_i \leftarrow c_i + a_j b_{(i-j) \bmod n}$ 
4:   end for
5: end for
```

3.2. Быстрое преобразование Фурье

Циклическую свертку можно выполнять при помощи дискретного преобразования Фурье, как было отмечено в разделе 2.3. Однако это неприменимо в криптографическом контексте, из-за того что дискретное преобразование Фурье выполняется в поле комплексных чисел, и операции с числами с плавающей точкой не поддаются анализу так же хорошо, как целые числа.

4. Обращения многочленов в $\frac{F_p[x]}{x^n-1}$

Генерация ключей требует нахождения обратного элемента $p^{-1} \in \frac{F_q[x]}{x^n-1}$, то есть требуется найти такой многочлен v , что:

$$p \cdot v \equiv 1 \pmod{x^n - 1}. \quad (12)$$

Это самая вычислительно дорогая операция. Основных алгоритмов два: расширенный алгоритм Евклида и подход, использующий малую теорему Ферма.

4.1. Расширенный алгоритм Евклида в $F_q[x]$

Классический расширенный алгоритм Евклида для многочленов над полем F_q находит НОД двух многочленов $a, b \in F_q[x]$ вместе с коэффициентами Безу s, t такими, что

$$s \cdot a + t \cdot b = \gcd(a, b). \quad (13)$$

Применяя расширенный алгоритм Евклида к $a = x^n - 1$ и $b = p(x)$, получим

$$s \cdot (x^n - 1) + t \cdot p(x) = \gcd(x^n - 1, p(x)). \quad (14)$$

Теперь возьмем полученное тождество по модулю $(x^n - 1)$:

$$t \cdot p(x) \equiv \gcd(x^n - 1, p(x)) \pmod{x^n - 1}. \quad (15)$$

Таким образом, обратный элемент существует тогда и только тогда, когда $\gcd(x^n - 1, p(x)) = c$, $c \in F_q$, и в этом случае

$$t \cdot p(x) \equiv c \pmod{x^n - 1} \quad (16)$$

$$p^{-1}(x) = c^{-1}t(x) \pmod{x^n - 1} \quad (17)$$

Ниже приведена реализация алгоритма в псевдокоде:

Algorithm 5 Расширенный алгоритм Евклида для нахождения p^{-1} , $p \in \mathbb{F}_q[x]/(x^n - 1)$

```

1:  $a \leftarrow x^n - 1$ 
2:  $b \leftarrow p$ 
3:  $v_{\text{old}} \leftarrow 0$ 
4:  $v \leftarrow 1$ 
5: loop
6:   Вычислить частное  $q \leftarrow \lfloor a/b \rfloor$  и остаток  $a \leftarrow a - q \cdot b$ 
7:   if  $a = 0$  then
8:     return  $b^{-1} \cdot v \bmod (x^n - 1)$   $\triangleright b$  — константа,  $v$  — искомый
      обратный
9:   end if
10:   $v_{\text{new}} \leftarrow v_{\text{old}} - q \cdot v \bmod (x^n - 1)$ 
11:   $(a, b) \leftarrow (b, a)$ 
12:   $(v_{\text{old}}, v) \leftarrow (v, v_{\text{new}})$ 
13: end loop

```

Несколько деталей реализации:

- Деление $\lfloor a/b \rfloor$ на шаге 6 выполняется стандартным делением «уголком» в $\mathbb{F}_q[x]$: на каждой итерации внутреннего цикла старший коэффициент a обнуляется вычитанием подходящего кратного b , используя таблицу деления в $\mathbb{F}_q$.
- Произведение $q \cdot v \bmod (x^n - 1)$ на шаге 10 реализовано циклической сверткой.

4.2. Нахождение обратного элемента через малую теорему Ферма

Пусть $R = \frac{\mathbb{F}_q[x]}{(x^n-1)}$. Разложим $(x^n - 1)$ в произведение неприводимых многочленов $f_1, f_2, \dots, f_l$ по китайской теореме остатков:

$$R = \frac{\mathbb{F}_q[x]}{f_1} \oplus \frac{\mathbb{F}_q[x]}{f_2} \oplus \dots \oplus \frac{\mathbb{F}_q[x]}{f_l} \quad (18)$$

Где каждое из $\frac{\mathbb{F}_q[x]}{f_i}$ изоморфно полю $F_{q^{\deg(f_i)}}$. Тогда в этих полях выполняется малая теорема Ферма

$$a^{q^{\deg(f_i)}-1} = 1 \quad (19)$$

И, эквивалентно:

$$a^{q^{\deg(f_i)}-2} = a^{-1} \quad (20)$$

Тогда в кольце R по китайской теореме об остатках будет справедливо

$$a^{lcm(q^{\deg(f_1)}-2, q^{\deg(f_2)}-2, \dots, q^{\deg(f_l)}-2)} = a^{-1} \quad (21)$$

Обозначим за S степень $lcm(q^{\deg(f_1)} - 2, q^{\deg(f_2)} - 2, \dots, q^{\deg(f_l)} - 2)$.

Поскольку показатель S экспоненциально велик, наивное возведение в степень неприемлемо. Применяется метод *быстрого возведения в степень*: S записывается в двоичном виде $S = \sum_{i=0}^k S_i 2^i$, и результат вычисляется за $O(\log S)$ умножений в R .

4.3. Сравнение с расширенным алгоритмом Евклида

Метод на основе малой теоремы Ферма требует $O(\log S)$ умножений многочленов (циклических сверток) в R , каждое из которых занимает $O(n^2)$ операций в $\mathbb{F}_q$ (или $O(n \log n)$ с использованием быстрого преобразования Фурье). Экспериментально было показано, что расширенный алгоритм Евклида выполняет порядка $O(n)$ итераций с $O(n)$ операциями каждая, то есть $O(n^2)$ суммарно. Таким образом, расширенный алгоритм Евклида предпочтительнее на практике.

5. Оптимизация циклических сверток в $\frac{F_4[x]}{x^n-1}$

В разделе 3 были показаны некоторые общие оптимизации циклических сверток, однако действительно значимый прирост можно получить только используя все возможности современных процессоров. Для этого можно представить свертку в F_4 , как несколько сверток в F_2 , для которых уже существуют оптимизированные реализации.

Представим $a \in F_4$ как два элемента из F_2 , a_r и a_i , $a = a_r + za_i$. Тогда при умножении двух элементов получим:

$$ab = (a_rb_r + z^2a_ib_i) + z(a_rb_i + a_ib_r) = (a_rb_r + a_ib_i) + z(a_rb_i + a_ib_r + a_ib_i) \quad (22)$$

Так, можно заметить что умножение в F_4 можно выполнить за счёт четырех умножений в F_2 . Число умножений можно сократить до трех, воспользовавшись алгоритмом Карацубы.

Существует много оптимизированных алгоритмов, осуществляющих свертки в F_2 . В данной работе была использована реализация из ВКЕ.

6. Оптимизация обращений многочленов из $\frac{F_4[x]}{x^n-1}$

В разделе 4 был сделан вывод о предпочтительности расширенного алгоритма Евклида для обращения многочлена. Однако у F_4 есть свойства, которые дают возможность реализовать значительно более быстрый алгоритм. Так, операция возведения в квадрат в $\mathbb{F}_4[x]/(x^n - 1)$ сводится к перестановке коэффициентов (циклическому сдвигу), аналогично случаю $\mathbb{F}_2$, поскольку $(a + b\alpha)^2 = a^2 + b^2\alpha^2 = a^2 + b^2(\alpha + 1)$, что вычисляется дешевле, чем полное умножение. Это позволяет заменить значительную часть умножений в алгоритме быстрого возведения в степень операциями k -возведения в квадрат, следуя схеме алгоритма, использованного авторами ВІКЕ [1].

Утверждение о показателе S для $\mathbb{F}_4$. Важным условием для применения алгоритма [1] является показатель степени S , в который возводится многочлен. К счастью, для простого n показатели для F_4 и F_2 полностью совпадают, что позволяет перенести алгоритм без изменения количества умножений.

Покажем, почему показатели совпадают: пусть n — простое число, выбранное так, что $x^n - 1$ раскладывается в произведение трёх неприводимых многочленов над $\mathbb{F}_4$: $x^n - 1 = f_1 f_2 f_3$ с $\deg f_i = d_i$. Тогда показатель степени S в кольце $\mathbb{F}_4[x]/(x^n - 1)$ совпадает с показателем, возникающим в случае $\mathbb{F}_2$:

$$S = \text{lcm}(4^{d_1} - 2, 4^{d_2} - 2, 4^{d_3} - 2) = 2^{n-1} - 2. \quad (23)$$

Таким образом, при корректном выборе n , алгоритм [1] из ВІКЕ переносится на случай F_4 , с некоторыми изменениями, которые не оказывают существенного влияния на скорость выполнения.

7. Результаты численных экспериментов

В криптосистеме ВІКЕ для достижения 192 бит классической стойкости или же 96 бит пост-квантовой безопасности (с учетом алгоритма Гровера) параметр n был выбран равным 12323. Самыми дорогими операциями являются взятие обратного элемента в факторкольце $\frac{\mathbb{F}_q[x]}{x^n-1}$ (нужно в KeyGen для публичного ключа $H_1^{-1}H$), а также циклические свертки (произведение циркулянтов). Реализация выполнена на языке C++ и доступна в [9].

KeyGen, Encaps и Decaps На языке C++ были реализованы алгоритмы KeyGen, Encaps и Decaps в двух вариантах:

- Наивная реализация для полей с порядком, равным произвольному простому числу.
- Ускоренная реализация для поля F_4 .

Параметры криптосистемы: $n = 12323$, $\gamma = 71$, $t = 132$. Количество итераций декодера: 1. Параметры криптосистемы ВІКЕ Level 3: $n = 24659$, $\gamma = 103$, $t = 199$. Количество итераций декодера: 4. *Важно отметить, что поскольку в реализованной криптосистеме используется F_4 , а в ВІКЕ используется F_2 . Поэтому для корректного сравнения в таблице параметр $n = 12323$ для криптосистемы над F_4 был умножен на 2 (24646).*

Обе программы были собраны с максимальным уровнем оптимизации и с распараллеливанием с использованием инструкций AVX2. В таблице 1 приведено сравнение базовых операций между реализацией от авторов ВІКЕ и реализацией в данной работе (GF4X). Операции gf2x_mod_inv и gf4x_mod_inv – обращение многочлена, а gf2x_mod_mul и gf4x_mod_mul – умножение многочленов (циклическая свертка).

В таблице 2 приведено сравнение производительности трёх основных операций КЕМ: KeyGen, Encaps и Decaps между реализацией ВІКЕ, реализацией над GF4, а также для наивной реализации.

Таблица 1. Сравнение производительности между операциями над F_2 из BIKE Level 3, и реализацией этих же операций для F_4

Алгоритм	N	Операция	Время выполнения (мс)
BIKE Level 3	24659	gf2x_mod_inv	12.070
		gf2x_mod_mul	0.571
GF4X	24646	gf4x_mod_inv	9.551
		gf4x_mod_mul	0.536

Таблица 2. Сравнение производительности между BIKE Level 3, реализацией для F_4 и наивной реализацией (GFPX, с $P = 5$)

Алгоритм	N	Операция	Время выполнения (мс)
BIKE Level 3	24659	Keygen	12.717
		Encaps	0.730
		Decaps	12.419
GF4X	24646	Keygen	12.587
		Encaps	0.721
		Decaps	4.607
GFPX	24646	Keygen	2009
		Encaps	829
		Decaps	22

Важно отметить, что декодер, используемый в BIKE Level 3 выполняет 4 итерации, в то время как декодер из GF4X выполняет 1 итерацию. Поэтому для корректного сравнения нужно рассматривать значение $12.419/4 = 3.10475$ для Decaps из BIKE Level 3.

8. Выводы

Полученная реализация криптосистемы для полей F_4 сравнима в производительности по сравнению с реализацией от авторов BIKE для F_2 . Эта реализация может иметь более хороший параметр DFR [2], а также может быть более сложной для взлома, ведь быстрых алгоритмов для F_2 больше чем для F_4 .

Список литературы

- [1] Nir Drucker, Shay Gueron и Dusan Kostic. «Fast polynomial inversion for post quantum QC-MDPC cryptography». В: *Information and Computation* 281 (2021), с. 104799. ISSN: 0890-5401. DOI: <https://doi.org/10.1016/j.ic.2021.104799>. URL: <https://www.sciencedirect.com/science/article/pii/S0890540121001152>.
- [2] Yu. S. Kosolapov K.Vedenev. «Theoretical analysis of decoding failure rate of non-binary QC-MDPC codes». В: *IACR Cryptology ePrint Archive* (2023).
- [3] NIST. *Post-Quantum Cryptography Standardization*. URL: <https://www.nist.gov/pqcrypto> (дата обр. 14.04.2026).
- [4] NIST. *PQC Standardization Process: Announcing Four Candidates to be Standardized, Plus Fourth Round Candidates*. 2022. URL: <https://csrc.nist.gov/News/2022/pqc-candidates-to-be-standardized-and-round-4> (дата обр. 14.05.2026).
- [5] NIST. *Recommendations for Key-Encapsulation Mechanisms*. 2025. URL: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-227.pdf> (дата обр. 14.05.2026).
- [6] NIST. *Status Report on the Fourth Round of the NIST Post-Quantum Cryptography Standardization Process*. 2025. URL: <https://www.nist.gov/pqcrypto> (дата обр. 14.04.2026).
- [7] BIKE team. *Официальный сайт криптосистемы BIKE*. URL: <https://bikesuite.org> (дата обр. 14.04.2026).
- [8] BIKE team. *Программная реализация BIKE за константное время*. URL: <https://github.com/awslabs/bike-kem> (дата обр. 14.04.2026).
- [9] *Исходный код реализации*. URL: https://github.com/kawaii-Code/qcmdpc_gf4 (дата обр. 16.05.2026).